

ORACLE
CloudWorld

Your Local LLM Web App in **OKE**



100% Data Privacy

Kosseila Haddalene
CloudThrill



Hi, I'm Kosseila

Cloud Dude



 Cloudthrill.ca

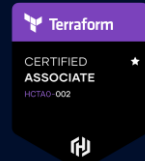
 Twitter.com/CloudDude_

 LinkedIn.com/in/kousshd

 GitHub.com/brokedba

- I live in Toronto  
- ~17 years infra (Oracle ACE pro)
- Blogger/Speaker
- IaC/DevOps enthusiast
- Founder, @CloudThrill
- Podcaster, Beatmaker   

Kosseila Hd Principal Consultant @CloudThrill



Hi, I'm Kosseila

Cloud Dude



 Cloudthrill.ca

 Twitter.com/CloudDude_

 LinkedIn.com/in/kousshd

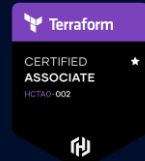
 GitHub.com/brokedba

- I live in Toronto 
- ~17 years infra (Oracle ACE pro)
- Blogger/Speaker
- IaC/DevOps enthusiast
- Founder, @CloudThrill
- Podcaster, Beatmaker   

Tech Beats Unplugged



Kosseila Hd Principal Consultant @CloudThrill



Hi, I'm Kosseila

Cloud Dude



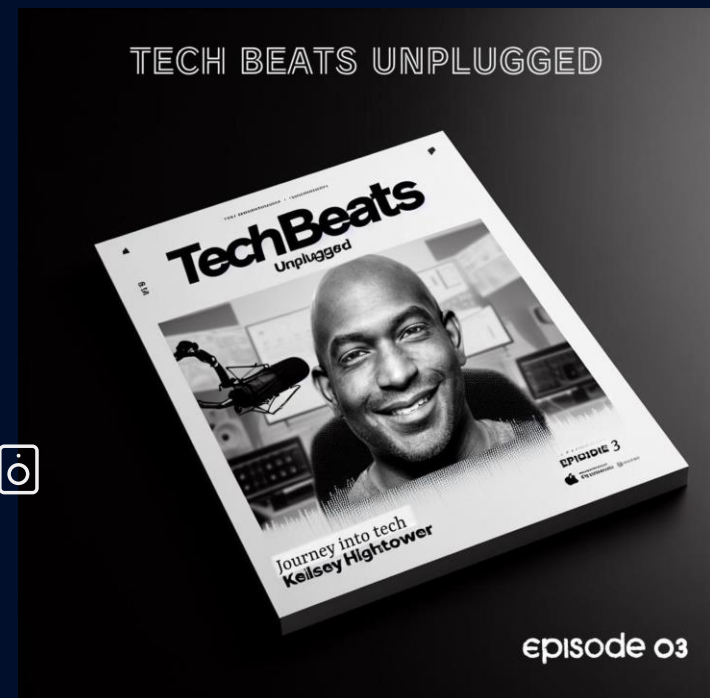
 Cloudthrill.ca

 Twitter.com/CloudDude_

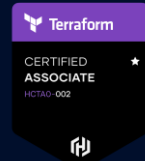
 LinkedIn.com/in/kousshd

 GitHub.com/brokedba

- I live in Toronto 
- ~17 years infra (Oracle ACE pro)
- Blogger/Speaker
- IaC/DevOps enthusiast
- Founder, @CloudThrill
- Podcaster, Beatmaker 



Kosseila Hd Principal Consultant @CloudThrill



Hi, I'm Kosseila

Cloud Dude



- I live in Toronto 🇧🇪🇨🇦
- ~17 years infra (Oracle ACE pro)
- Blogger/Speaker
- IaC/DevOps enthusiast
- Founder, @CloudThrill
- Podcaster, Beatmaker 🎧🎵🎧

 Cloudthrill.ca

 Twitter.com/CloudDude_

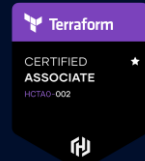
 LinkedIn.com/in/kousshd

 GitHub.com/brokedba

Tech Beats Unplugged



Kosseila Hd Principal Consultant @CloudThrill



AGENDA

1- OpenAI is not your friend!

- ❑ Risk of proprietary AI models
- ❑ Privacy: Your business is your primary asset

2- Introduction to Local LLMs

- ❑ Benefits over proprietary models.
- ❑ LLMs: WHAT ,WHERE ,HOW
- ❑ Inference Engines: **OLLAMA**

3- Building a Private LLM Webapp in OKE

- ❑ Architecture challenges & Evolution :
PHASE1: Build web app from scratch (Python/Streamlit)
PHASE2: Existing popular WebUIs

4- Live Demo

- ❑ Local implementation with Streamlit & Ollama server
- ❑ Deploy Ollama/OpenWebUI in OKE using Helm charts.

5- Q&A



Scan me

AGENDA

1- OpenAI is not your friend!

- ❑ Risk of proprietary AI models
- ❑ Privacy: Your business is your primary asset

2- Introduction to Local LLMs

- ❑ Benefits over proprietary models.
- ❑ LLMs: WHAT ,WHERE ,HOW
- ❑ Inference Engines: **OLLAMA**

3- Building a Private LLM Webapp in OKE

- ❑ Architecture challenges & Evolution :
PHASE1: Build web app from scratch (Python/Streamlit)
PHASE2: Existing popular WebUIs

4- Live Demo

- ❑ Local implementation with Streamlit & Ollama server
- ❑ Deploy Ollama/OpenWebUI in OKE using Helm charts.

5- Q&A

GitHub Repo



Scan me

OpenAI is Not Your Friend (The Risks)



Security

- Data sent through OpenAI API could be exposed.
- Data is exposed to 3rd-party subprocessors (snowflake).
- Prioritizes commercial success over safety (Data breach).

OpenAI is Not Your Friend (The Risks)



Security

- Data sent through OpenAI API could be exposed.
- Data is exposed to 3rd-party subprocessors (snowflake).
- Prioritizes commercial success over safety (Data breach).

Data Privacy

- Loss of control over sensitive business data.
- Customer interaction data used to train their models.
- GDPR compliance uncertain.

OpenAI is Not Your Friend (The Risks)



Model	Pricing	Pricing with Batch API*
gpt-4o	\$5.00 / 1M input tokens	\$2.50 / 1M input tokens
	\$15.00 / 1M output tokens	\$7.50 / 1M output tokens
gpt-4o-2024-08-06	\$2.50 / 1M input tokens	\$1.25 / 1M input tokens
	\$10.00 / 1M output tokens	\$5.00 / 1M output tokens
gpt-4o-2024-05-13	\$5.00 / 1M input tokens	\$2.50 / 1M input tokens
	\$15.00 / 1M output tokens	\$7.50 / 1M output tokens

Security

- Data sent through OpenAI API could be exposed.
- Data is exposed to 3rd-party subprocessors (snowflake).
- Prioritizes commercial success over safety (Data breach).

Data Privacy

- Loss of control over sensitive business data.
- Customer interaction data used to train their models.
- GDPR compliance uncertain.

Cost (in/out tk)

- Expensive subscription and usage fees over time.
- You pay for input and output tokens.
- Pricing model can change anytime.

OpenAI is Not Your Friend (The Risks)



Security

- Data sent through OpenAI API could be exposed.
- Data is exposed to 3rd-party subprocessors (snowflake).
- Prioritizes commercial success over safety (Data breach).

Data Privacy

- Loss of control over sensitive business data.
- Customer interaction data used to train their models.
- GDPR compliance uncertain.

Cost (in/out tk)

- Expensive subscription and usage fees over time.
- You pay for input and output tokens.
- Pricing model can change anytime.

Transparency Issues

- Black-box nature of proprietary models = hard to assess potential risks.
- limited fine tuning of the parameters.
- 0 details about its training

OpenAI is Not Your Friend (The Risks)

Companies That Have Banned ChatGPT



Verizon



Amazon



Samsung



Citigroup



Spotify



Apple



Bank of America



Wells Fargo



Northrop Grumman



Microsoft



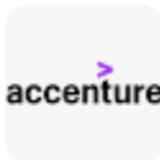
JPMorgan Chase



Deutsche Bank



Goldman Sachs



Accenture



Commonwealth Bank



Privacy: Your data is your primary Asset. Protect it?

Data Sovereignty

Keep your data within your infrastructure. Your data is at stake.

Regulatory Compliance

Adherence to data protection regulations (GDPR, HIPAA,CCPA). Some sectors forbid the use of chatGPT

Enhanced Security

Mitigate the risk of data breaches by maintaining control.



Open LLMs: Future of Customizable AI



Open LLMs: Future of Customizable AI

1

Definition & Benefits

Open-source Large Language Models (LLMs) are free AI models that can be customized and modified to suit specific business needs. They offer unparalleled transparency, allowing developers to understand and improve the underlying algorithms.



Open LLMs: Future of Customizable AI

1

Definition & Benefits

Open-source Large Language Models (LLMs) are free AI models that can be customized and modified to suit specific business needs. They offer unparalleled transparency, allowing developers to understand and improve the underlying algorithms.

2

Where to Find/Use Open LLMs

Platforms like Hugging Face host thousands of pre-trained models, while GitHub repositories provide community-contributed LLMs. Specialized libraries such as Model Zoo offer task-specific AI models for various applications.



Open LLMs: Future of Customizable AI

1

Definition & Benefits

Open-source Large Language Models (LLMs) are free AI models that can be customized and modified to suit specific business needs. They offer unparalleled transparency, allowing developers to understand and improve the underlying algorithms.

2

Where to Find/Use Open LLMs

Platforms like Hugging Face host thousands of pre-trained models, while GitHub repositories provide community-contributed LLMs. Specialized libraries such as Model Zoo offer task-specific AI models for various applications.

3

Inference Engines

Tools like Ollama, vLLM, localAI, and LMStudio provide efficient ways to run and deploy open LLMs. These engines offer varying degrees of user-friendliness, resource efficiency, and developer tools to suit different project requirements.

Open LLMs: What, Where, How



WHAT

Definition:

Open LLMs are freely available language models for customization.

Advantages:

Modifiable, fully transparent, and cost-effective

Open LLMs: What, Where, How



WHAT

Definition:

Open LLMs are freely available language models for customization.

Advantages:

Modifiable, fully transparent, and cost-effective



Lamma is FREE but not Open-source unlike Mistral



Models

Base Model

- Purpose: Text completion without QA fine-tuning.
- Behavior: Ideal for predictive text but lacks conversational fluidity.



Models

Base Model

- Purpose: Text completion without QA fine-tuning.
- Behavior: Ideal for predictive text but lacks conversational fluidity.

Instruct Model

- Purpose: Fine-tuned for single-turn interactions (e.g., code generation).
- Behavior: Directly answers questions, but struggles with multi-turn conversations.

A background image for the slide featuring a server rack with glowing blue lights and red smoke or smoke-like patterns rising from the top and bottom. The letters 'LLM' are overlaid in large, white, bold font on the left side of the image.

Models

Base Model

- Purpose: Text completion without QA fine-tuning.
- Behavior: Ideal for predictive text but lacks conversational fluidity.

Instruct Model

- Purpose: Fine-tuned for single-turn interactions (e.g., code generation).
- Behavior: Directly answers questions, but struggles with multi-turn conversations.

Chat Model

- Purpose: Designed for coherent multi-turn dialogues.
- Behavior: Retains context across exchanges, ideal for chatbots.

The background of the slide features a dark, atmospheric image of server racks in a data center. The racks are illuminated with a mix of blue and red lights, creating a sense of depth and technology. In the foreground, the large, white, bold letters 'LLM' are superimposed on the left side. The overall aesthetic is futuristic and tech-oriented.

LLM

Model Quantization

- Quantization reduces model size by using fewer bits per parameter for resource-limited devices.
- a 32-bit model requires 4GB of memory for every 1 Billion parameters.

Types & Formats:

f32, INT8, fixed points

Basic quantization with some performance loss (f32/8bits).

Integer Quants (Q4_0, Q5_0)

- x-bit integer quantization without scaling factors

New K-Quants (Q3_K, Q5_K)

Smarter bit allocation for better size-performance balance (accuracy loss).

LLM

Model Quantization

- Quantization reduces model size by using fewer bits per parameter for resource-limited devices.
- a 32-bit model requires 4GB of memory for every 1 Billion parameters.

Types & Formats:

f32, INT8, fixed points

Basic quantization with some performance loss (f32/8bits).

Integer Quants (Q4_0, Q5_0)

- x-bit integer quantization without scaling factors

New K-Quants (Q3_K, Q5_K)

Smarter bit allocation for better size-performance balance (accuracy loss).

Perplexity

Metric for model prediction accuracy—lower perplexity means better text generation performance.

LLM

Model Quantization

- Quantization reduces model size by using fewer bits per parameter for resource-limited devices.
- a 32-bit model requires 4GB of memory for every 1 Billion parameters.

Types & Formats:

f32, INT8, fixed points

Basic quantization with some performance loss (f32/8bits).

Integer Quants (Q4_0, Q5_0)

- x-bit integer quantization without scaling factors

New K-Quants (Q3_K, Q5_K)

Smarter bit allocation for better size-performance balance (accuracy loss).

Perplexity

There's also MMLU, HumanEval, GQPA.

Metric for model prediction accuracy—lower perplexity means better text generation performance.

Tools & Libraries (CPU compatible)

llama.cpp

Purpose: C++ library for efficient LLM inference on both CPU and GPU.

Format: Supports GGUF with various quantization options.

Deployment: Ideal for local hardware and cloud environments.

GGML & GGUF

- GGML: Optimized for CPU inference.
- GGUF: Future-proof format with better tokenization and metadata support.

CPU-Based Models:

- **Use:** Efficient CPU-only inference for models like LLaMA, GPT-4All, and Mistral.
- **Tool:** llama.cpp leverages GGUF format for compact storage and fast inference.

GGUF (GPT-Generated Unified Format): Optimized for faster CPU-based inference or GPU acceleration using llama.cpp and other libraries. Replacing GGML.

K Quant Naming Convention: "Q[Number]_K_Size"

Meta-Llama-3-8B-Q4_K_M-GGUF

Tools & Libraries (CPU compatible)

llama.cpp

Purpose: C++ library for efficient LLM inference on both CPU and GPU.

Format: Supports GGUF with various quantization options.

Deployment: Ideal for local hardware and cloud environments.

GGML & GGUF

- GGML: Optimized for CPU inference.
- GGUF: Future-proof format with better tokenization and metadata support.

CPU-Based Models:

- **Use:** Efficient CPU-only inference for models like LLaMA, GPT-4All, and Mistral.
- **Tool:** llama.cpp leverages GGUF format for compact storage and fast inference.

GGUF (GPT-Generated Unified Format): Optimized for faster CPU-based inference or GPU acceleration using llama.cpp and other libraries. Replacing GGML.

K Quant Naming Convention: "Q[Number]_K_Size"

Meta-Llama-3-8B-Q4_K_M-GGUF

"Q[Number]": bit depth.

Tools & Libraries (CPU compatible)

llama.cpp

Purpose: C++ library for efficient LLM inference on both CPU and GPU.

Format: Supports GGUF with various quantization options.

Deployment: Ideal for local hardware and cloud environments.

GGML & GGUF

- GGML: Optimized for CPU inference.
- GGUF: Future-proof format with better tokenization and metadata support.

CPU-Based Models:

- **Use:** Efficient CPU-only inference for models like LLaMA, GPT-4All, and Mistral.
- **Tool:** llama.cpp leverages GGUF format for compact storage and fast inference.

GGUF (GPT-Generated Unified Format): Optimized for faster CPU-based inference or GPU acceleration using llama.cpp and other libraries. Replacing GGML.

K Quant Naming Convention: "Q[Number]_K_Size"

Meta-Llama-3-8B-Q4_K_M-GGUF

"Q[Number]": bit depth.

"K": knowledge distillation.

Tools & Libraries (CPU compatible)

llama.cpp

Purpose: C++ library for efficient LLM inference on both CPU and GPU.

Format: Supports GGUF with various quantization options.

Deployment: Ideal for local hardware and cloud environments.

GGML & GGUF

- GGML: Optimized for CPU inference.
- GGUF: Future-proof format with better tokenization and metadata support.

CPU-Based Models:

- **Use:** Efficient CPU-only inference for models like LLaMA, GPT-4All, and Mistral.
- **Tool:** llama.cpp leverages GGUF format for compact storage and fast inference.

GGUF (GPT-Generated Unified Format): Optimized for faster CPU-based inference or GPU acceleration using llama.cpp and other libraries. Replacing GGML.

K Quant Naming Convention: "Q[Number]_K_Size"

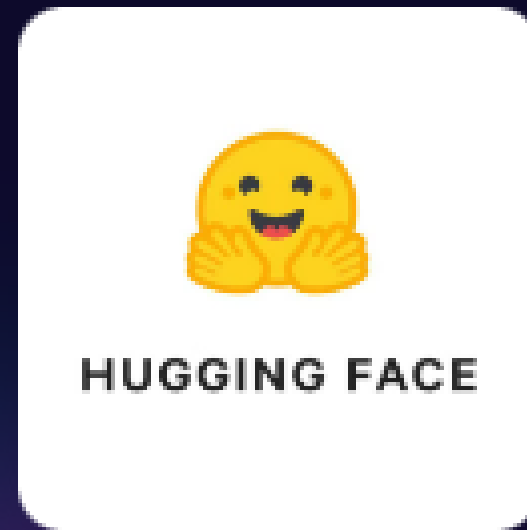
Meta-Llama-3-8B-Q4_K_M-GGUF

"Q[Number]": bit depth.

"K": knowledge distillation.

"Size" "S"mall, "M"edium, "L"arge.

Open LLMs:



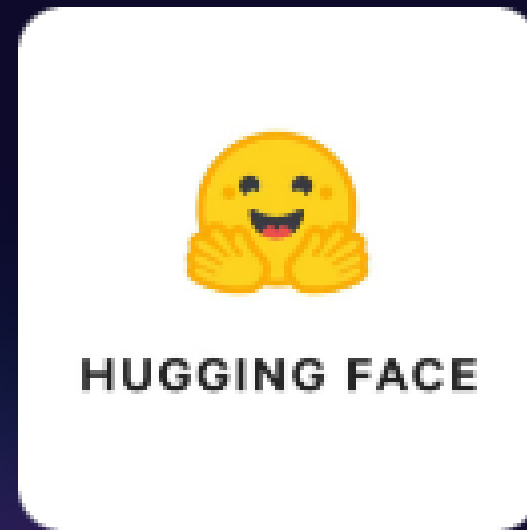
WHERE

Hugging Face: Platform with 1000s of pre-trained models for various tasks.

GitHub: Repo for community based models (+ Beta Marketplace).

CivitAI: GoTo Hub for Image generation models.

Open LLMs:



Features

Transformers
libraries(pytorch/tensorflow)

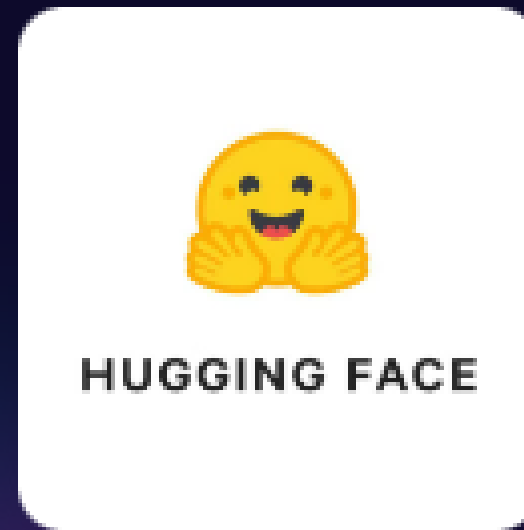
Tokenizer
converting words into tokens

Datasets Hub
Train & Fine-tune Models

Spaces/chat
Perform Inference

Pipeline
Pre-built workflows for common tasks

Open LLMs:



Features

Transformers
libraries(pytorch/tensorflow)

Tokenizer
converting words into tokens

Datasets Hub
Train & Fine-tune Models

Spaces/chat
Perform Inference

Pipeline
Pre-built workflows for common tasks

❑ You can check if your machine can run LLM in HuggingFace called ["CAN-IT-RUN-LLM"](#)



How

Online Access

Platforms like [HuggingChat](#) allow you to interact with LLMs directly via a web interface.

Local Inference

Download models, use PyTorch and transformers library to run them locally by downloading configuration, tokenizer, and model files.

Inference Engines

Use Engines like Ollama, Lolcalai or vLLM to run LLMs locally or via an AI API endpoint,

How

Online Access

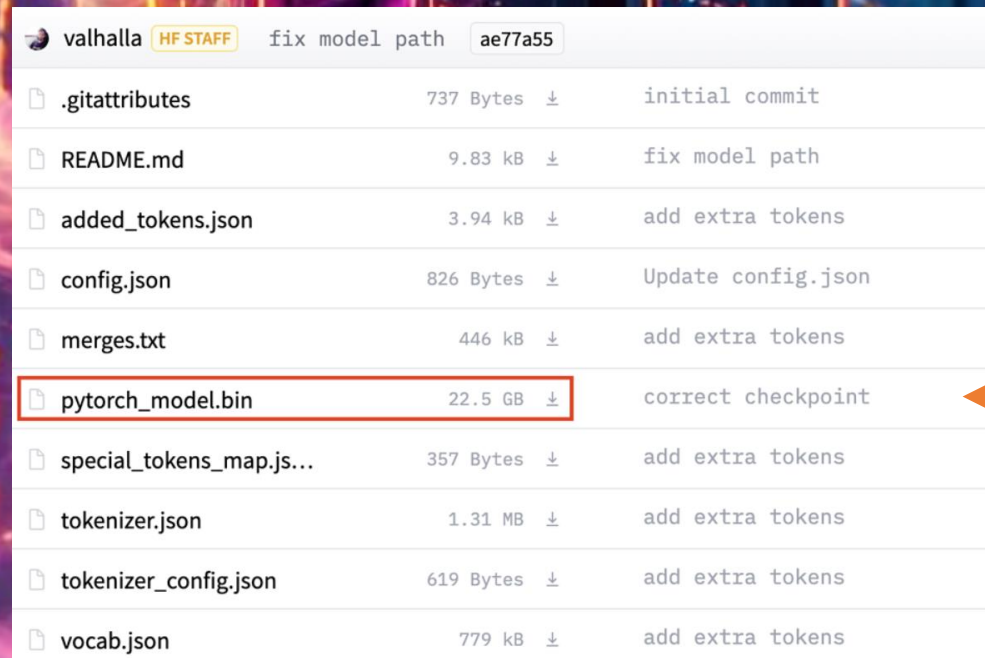
Platforms like [HuggingChat](#) allow you to interact with LLMs directly via a web interface.

Local Inference

Download models, use PyTorch and transformers library to run them locally by downloading configuration, tokenizer, and model files.

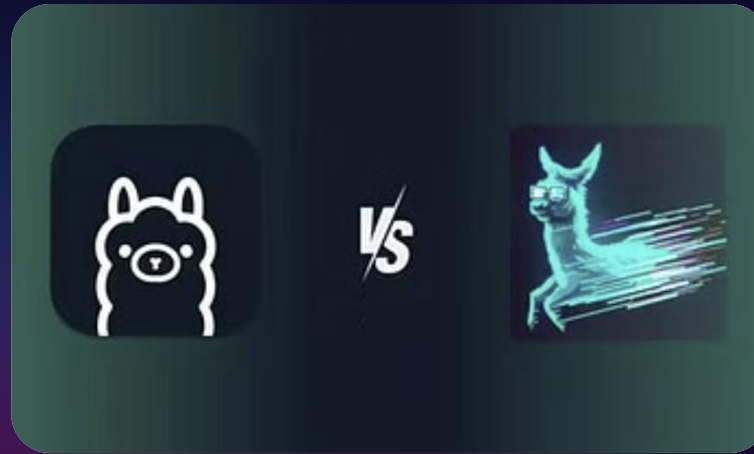
Inference Engines

Use Engines like Ollama, Lollalai or vLLM to run LLMs locally or via an AI API endpoint,



valhalla	HF STAFF	fix model path	ae77a55
.gitattributes	737 Bytes	↓	initial commit
README.md	9.83 kB	↓	fix model path
added_tokens.json	3.94 kB	↓	add extra tokens
config.json	826 Bytes	↓	Update config.json
merges.txt	446 kB	↓	add extra tokens
pytorch_model.bin	22.5 GB	↓	correct checkpoint
special_tokens_map.js...	357 Bytes	↓	add extra tokens
tokenizer.json	1.31 MB	↓	add extra tokens
tokenizer_config.json	619 Bytes	↓	add extra tokens
vocab.json	779 kB	↓	add extra tokens

Open LLMs: Inference Engines



- Ollama: Lightweight, user-friendly LLM engine.
- vLLM: Efficient with minimal resource usage.
- localAI: Flexible, supports diverse models.
- LMStudio: Developer-centric with extensive tools.

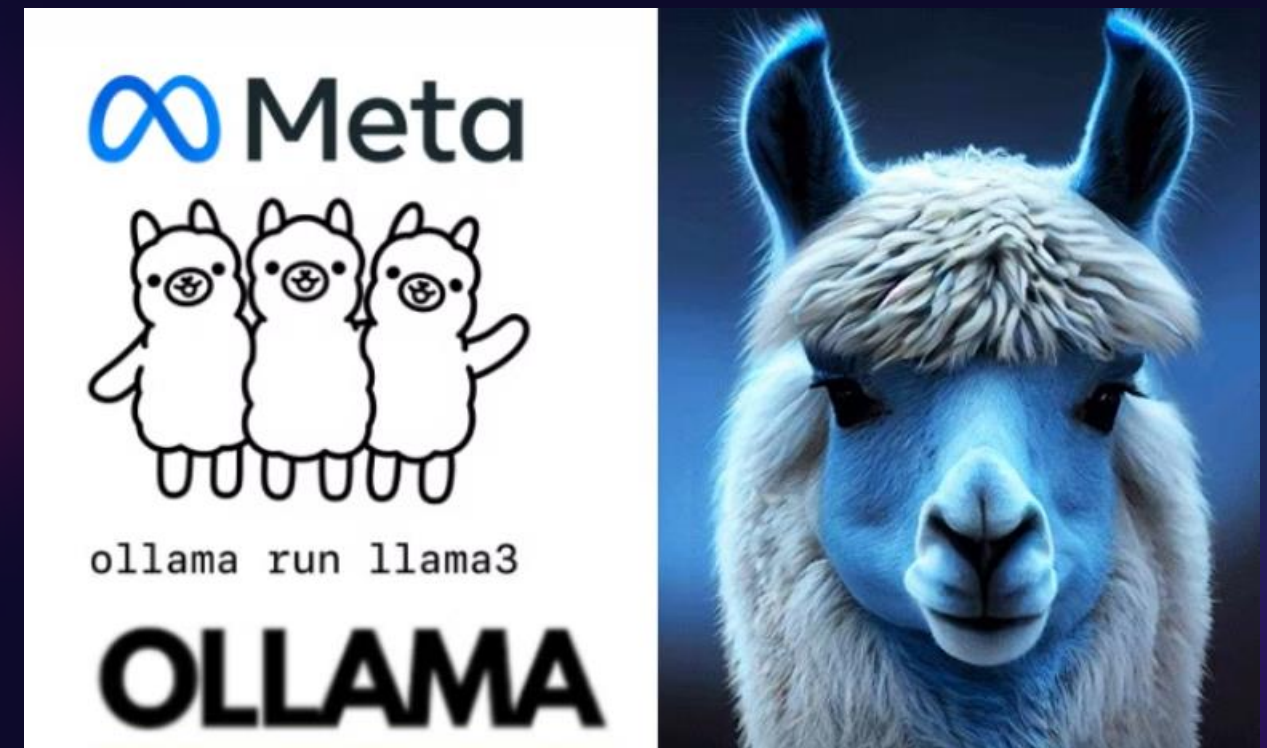
OLLAMA

Lightweight & User-Friendly:

- Designed for minimal resource consumption.
- Simple to set up and operate.
- Supports efficient inference with local models.

Benefits:

- Ideal for beginners and small-scale projects.
- Quickly deploy LLMs on your own hardware.



Ollama API & Environment Variables

API Methods

- ▶ `/api/generate`
For text completion (i.e summarization)
- ▶ `/api/chat`
For multi-turn conversations
- ▶ `/api/models`
To list, pull, update, or delete models.

Ollama listening port: 11434

Ollama Installation

1. Linux

```
curl -fsSL https://ollama.com/install.sh | sh
```

1. Docker

```
docker run -d -v ollama:/root/.ollama -p  
11434:11434 --name ollama ollama/ollama  
--gpus=all
```

Environment Variables

- OLLAMA_MODELS: Default model location.
- OLLAMA_NUM_PARALLEL: Number of parallel requests (.i.e, 3 requests).
- OLLAMA_MAX_LOADED_MODELS: Maximum loaded models at once (default 3)
- OLLAMA_MAX_QUEUE: Maximum request queue before rejection (512)
- OLLAMA_KEEP_ALIVE: Keeps model in memory (default: 5 minutes; -1 forever).
- OLLAMA_ORIGINS: Restrict allowed origins (default: listens on all interfaces).

Ollama Commands

- `serve`: Starts the Ollama server.

```
Usage:
  ollama [command]

Available Commands:
  serve      Start ollama
  create     Create a model from a Modelfile
  show       Show information for a model
  run        Run a model
  pull       Pull a model from a registry
  push       Push a model to a registry
  list       List models
  cp         Copy a model
  rm         Remove a model
  help       Help about any command
```

- Models location: `/usr/share/ollama/.ollama/models`
- Logs:
`~/.ollama/logs/server.log`
`journalctl -u ollama.service`
- Enable Debug: `OLLAMA_DEBUG="1"`
- [Ollama FAQ](#)

Ollama Commands

- `serve`: Starts the Ollama server.
- `show`: Shows information for a specific model.
- `list`: Lists all available models.

```
ollama list
```

```
Usage:
  ollama [command]

Available Commands:
  serve      Start ollama
  create     Create a model from a Modelfile
  show       Show information for a model
  run        Run a model
  pull       Pull a model from a registry
  push       Push a model to a registry
  list       List models
  cp         Copy a model
  rm         Remove a model
  help       Help about any command
```

- Models location: `/usr/share/ollama/.ollama/models`
- Logs:
`~/.ollama/logs/server.log`
`journalctl -u ollama.service`
- Enable Debug: `OLLAMA_DEBUG="1"`
- [Ollama FAQ](#)

Ollama Commands

- `serve`: Starts the Ollama server.
- `show`: Shows information for a specific model.
- `list`: Lists all available models.

```
ollama list
```

- `pull`: Downloads a model from the registry.

```
ollama pull <model_name>
```

- `run`: Executes a model on a given input.

```
ollama run llama2-uncensored
```

```
Usage:
  ollama [command]

Available Commands:
  serve      Start ollama
  create     Create a model from a Modelfile
  show       Show information for a model
  run        Run a model
  pull       Pull a model from a registry
  push       Push a model to a registry
  list       List models
  cp         Copy a model
  rm         Remove a model
  help       Help about any command
```

○ Models location: `/usr/share/ollama/.ollama/models`

○ Logs:

```
~/.ollama/logs/server.log
```

```
journalctl -u ollama.service
```

○ Enable Debug: `OLLAMA_DEBUG="1"`

○ [Ollama FAQ](#)

Ollama Commands

- **serve**: Starts the Ollama server.
- **show**: Shows information for a specific model.
- **list**: Lists all available models.

```
ollama list
```

- **pull**: Downloads a model from the registry.

```
ollama pull <model_name>
```

- **run**: Executes a model on a given input.

```
ollama run llama2-uncensored
```

- **create**: Builds a model from a Modelfile.

```
ollama create llama3.1 -f modelfile
```

```
Usage:
  ollama [command]

Available Commands:
  serve      Start ollama
  create     Create a model from a Modelfile
  show      Show information for a model
  run       Run a model
  pull      Pull a model from a registry
  push      Push a model to a registry
  list      List models
  cp        Copy a model
  rm        Remove a model
  help      Help about any command
```

○ Models location: `/usr/share/ollama/.ollama/models`

○ Logs:

```
~/.ollama/logs/server.log
```

```
journalctl -u ollama.service
```

○ Enable Debug: `OLLAMA_DEBUG="1"`

○ [Ollama FAQ](#)

Ollama Commands

- `serve`: Starts the Ollama server.
- `show`: Shows information for a specific model.
- `list`: Lists all available models.

```
ollama list
```

- `pull`: Downloads a model from the registry.

```
ollama pull <model_name>
```

- `run`: Executes a model on a given input.

```
ollama run llama2-uncensored
```

- `create`: Builds a model from a Modelfile.

```
ollama create llama3.1 -f modelfile
```

- `ps`: List running models

```
ollama ps
```

```
Usage:
  ollama [command]

Available Commands:
  serve      Start ollama
  create     Create a model from a Modelfile
  show       Show information for a model
  run        Run a model
  pull       Pull a model from a registry
  push       Push a model to a registry
  list       List models
  cp         Copy a model
  rm         Remove a model
  help       Help about any command
```

○ Models location: `/usr/share/ollama/.ollama/models`

○ Logs:

```
~/.ollama/logs/server.log
```

```
journalctl -u ollama.service
```

○ Enable Debug: `OLLAMA_DEBUG="1"`

○ [Ollama FAQ](#)

Modelfile

Modelfile is a config file that customizes model behavior by embedding prompts, adjusting parameters , and controlling the conversational styles and tone.

Import the model

Specify the location of the LLM file or the model ID.

Configure the model

Set parameters (temperature, top-k, and stop sequences).

Define prompt template

Specify a template for generating text using the LLM, including placeholders for user input.

Set system messages

Define custom system messages that can be used in the generated text.

Modelfile Key Instructions

Instruction	Description
<u>FROM</u> (required)	Defines the base model to use.
<u>PARAMETER</u>	Sets the parameters for how Ollama will run the model.
<u>TEMPLATE</u>	The full prompt template to be sent to the model.
<u>SYSTEM</u>	Specifies the system message that will be set in the template.
<u>ADAPTER</u>	Defines the (Q)LoRA adapters to apply to the model.
<u>LICENSE</u>	Specifies the legal license.
<u>MESSAGE</u>	Specify message history.

Modelfile

Instruction

FROM (required)

PARAMETER

TEMPLATE

SYSTEM

ADAPTER

LICENSE

MESSAGE

```
$ vi llama3.loc
```

```
# Set the base model
```

```
FROM ./ai_models/Meta-Llama-3-8B-Instruct.Q3_K_S.gguf
```

```
# Custom parameter values
```

```
PARAMETER temperature 1
```

```
PARAMETER stop <|start_header_id|>
```

```
PARAMETER stop <|end_header_id|>
```

```
PARAMETER stop <|eot_id|>
```

```
# Set the model template
```

```
TEMPLATE """
```

```
{{ if .System }}<|start_header_id|>system<|end_header_id|>.
```

```
{{ .System }}<|eot_id|>{{ end }}
```

```
{{ if .Prompt }}<|start_header_id|>user<|end_header_id|>
```

```
{{ .Prompt }}<|eot_id|>{{ end }}
```

```
<|start_header_id|>assistant<|end_header_id|>
```

```
{{ .Response }}<|eot_id|>
```

```
"""
```

```
# Set the system message
```

```
SYSTEM You are a helpful AI assistant named Llama3 Droid
```

Template

Variables

- `{{ .System }}`:
Custom system message
- `{{ .Prompt }}`:
User's input prompt
- `{{ .Response }}`:
Model's response

Special Tokens

- `<|begin_of_text|>`
The start of the prompt.
- `<|eot_id|>`
End of a message in a turn
- `<|start_header_id|>{role}`
`<|end_header_id|>`
msg roles (`system`, `user`, `assistant`)
- `<|end_of_text|>`:
Stops token generation (base model).

Message Roles

- **System**: System message
- **User**: Example of user input
- **Assistant**: Example of the model's response

 Note: Specific to Llama 3

Building Private LLM Chatbot in OKE

Architecture & Challenges

1

Deployment
Complexity

How to simplify the deployment
with automated tools and scripts.

2

Performance
Optimization

Optimize AI resource allocation for
efficient operations.

3

Data Privacy

Ensure complete data isolation
within your OKE environment.

Evolution of Design



1

Phase 1: Building from
Scratch

The initial approach Building the
web app from scratch. Python ,
Streamlit.

Evolution of Design



1

Phase 1: Building from Scratch

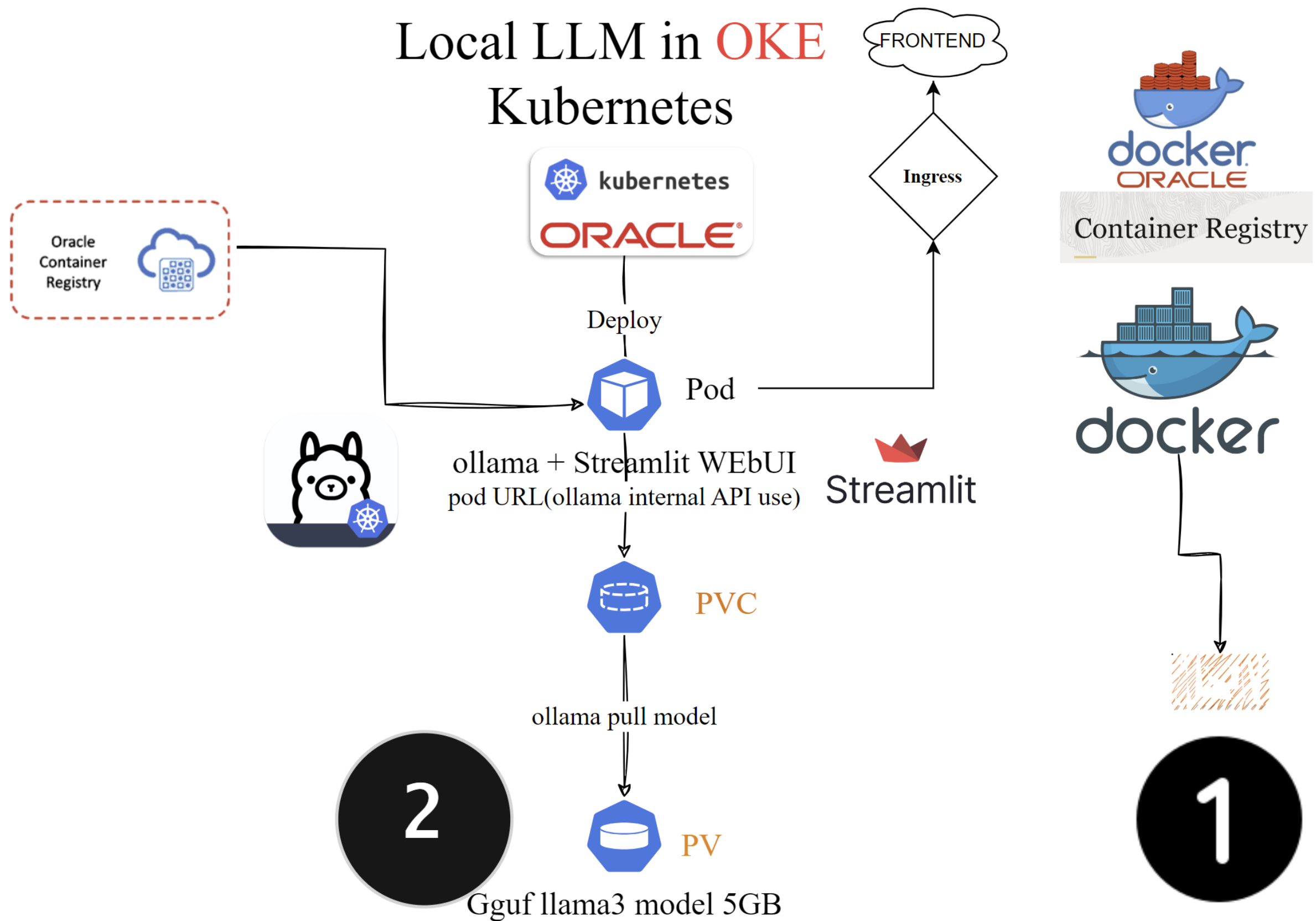
The initial approach Building the web app from scratch. Python, Streamlit.



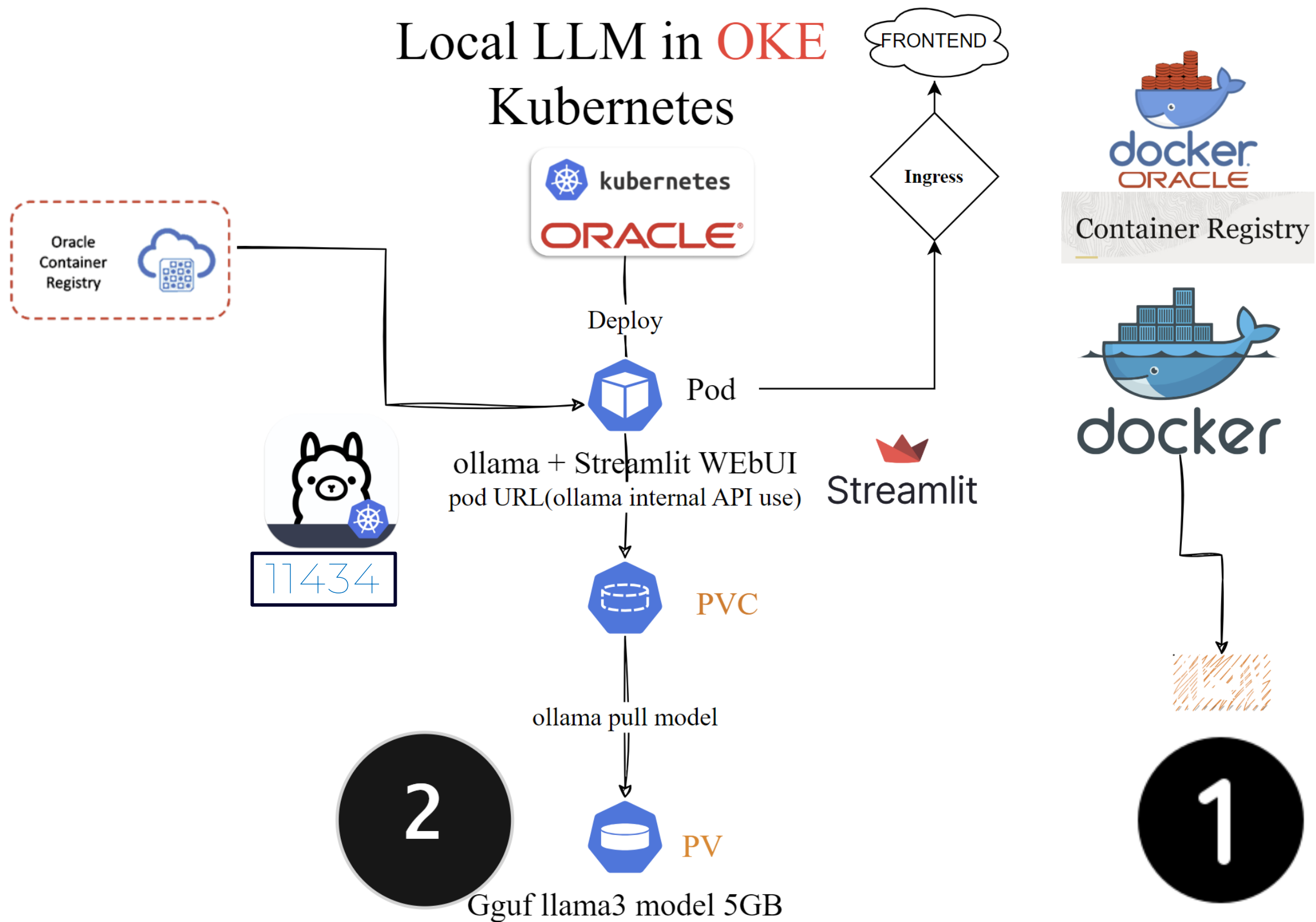
Standalone machine

- Streamlit for UI
- Python Front end
- Ollama for LLM backend.

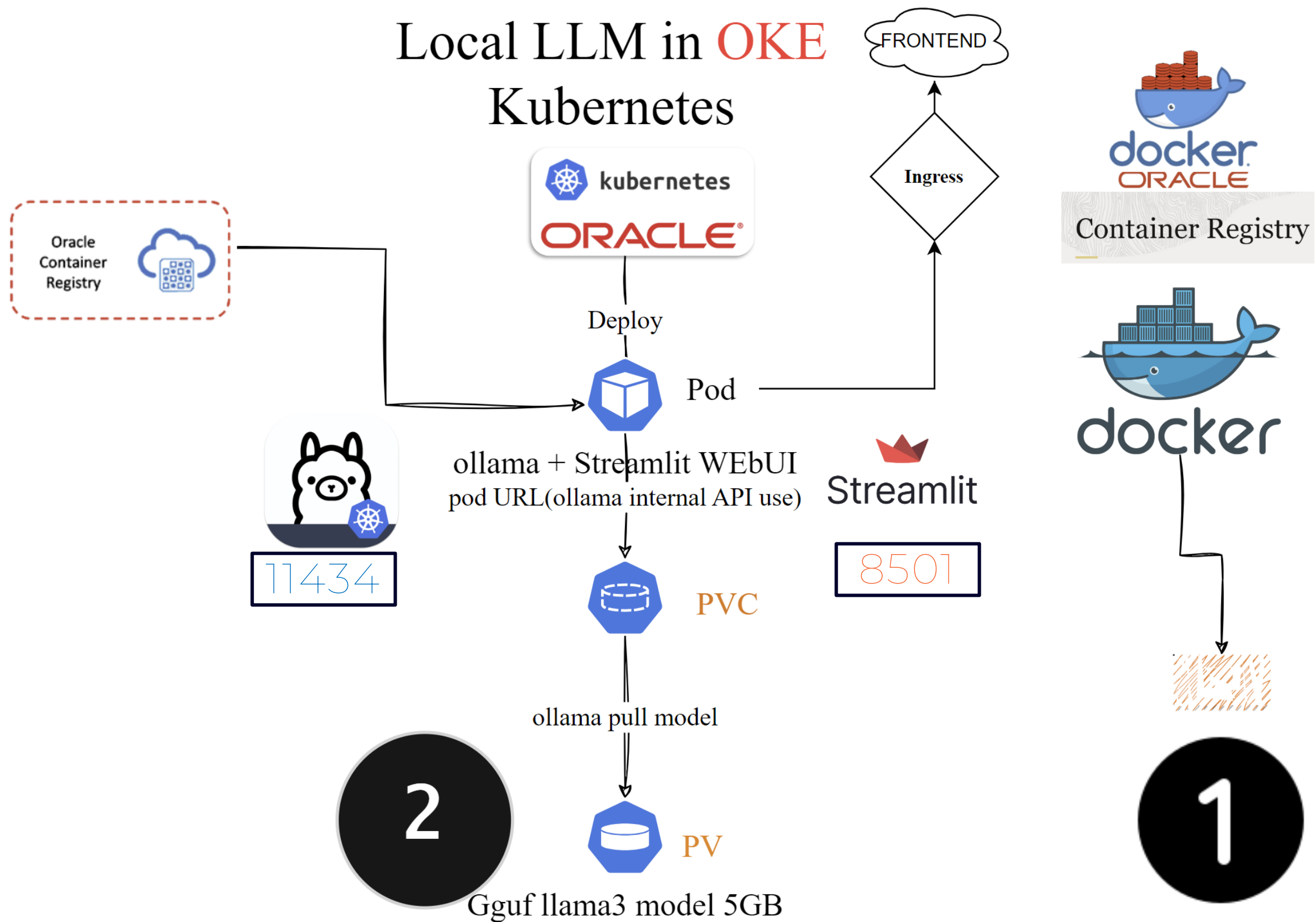
Local LLM in OKE Kubernetes



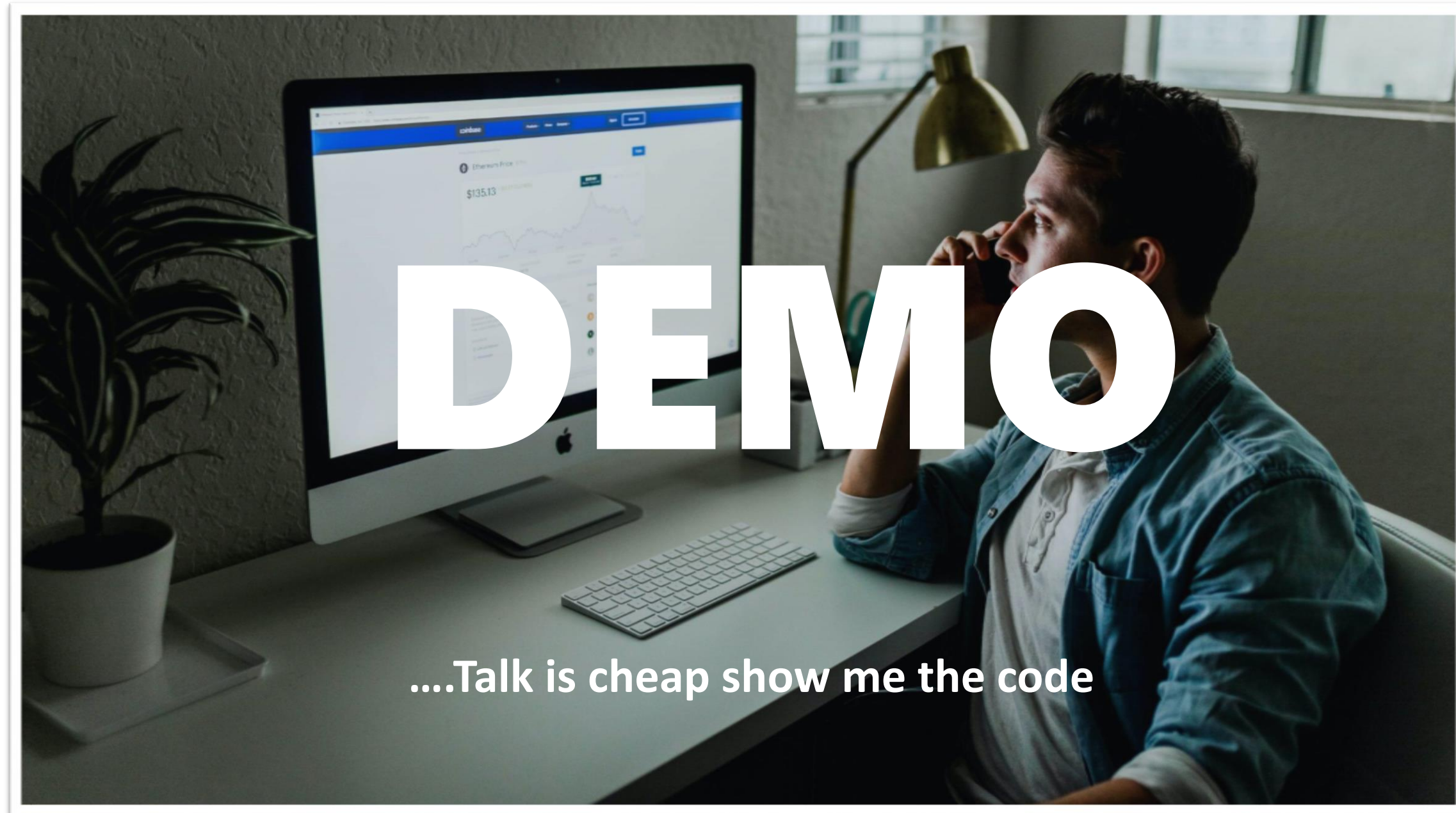
Local LLM in OKE Kubernetes



Local LLM in OKE Kubernetes



Local LLM Web App using Streamlit



....Talk is cheap show me the code

Evolution of Design



2

Phase 2: Integrating
Existing WebUIs

Integrating with existing popular
WebUIs.

Evolution of Design



2

Phase 2: Integrating
Existing WebUIs

Integrating with existing popular
WebUIs.



LobeChat



Chatbot UI



Chatbot Ollama

Alpaca WebUI



ChatOllama



LibreChat



OpenWebUI

Evolution of Design



2

Phase 2: Integrating
Existing WebUIs

Integrating with existing popular
WebUIs.



OCI Kubernetes engine

- 2 pods: Helm charts for deployment.
- Ollama backend, initcontainer
- OpenWebUI

Evolution of Design



2

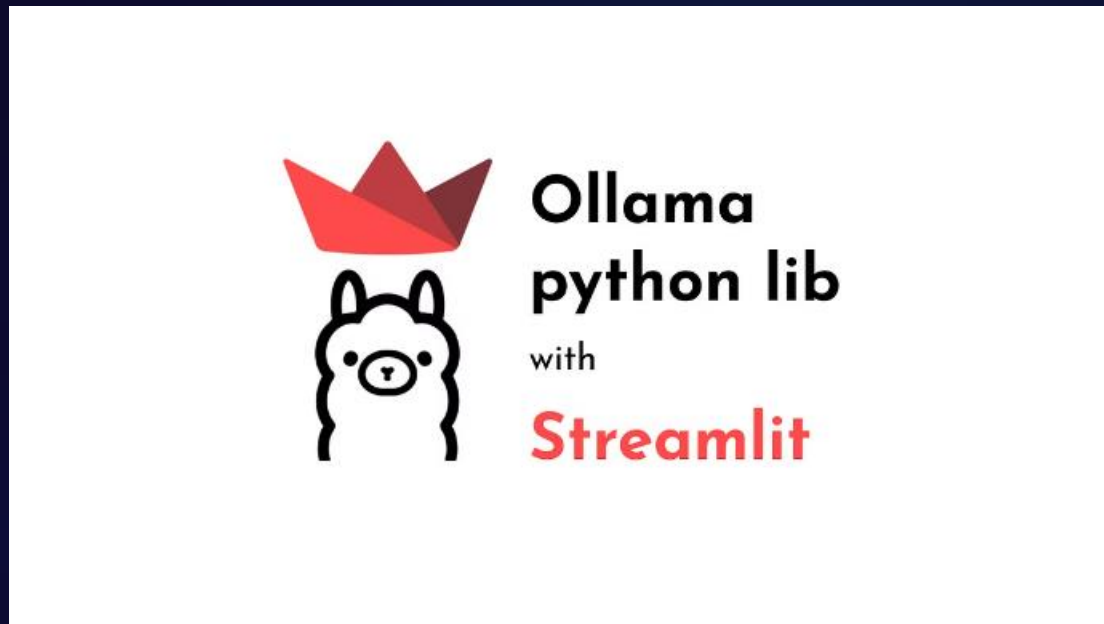
Phase 2: Integrating
Existing WebUIs

Integrating with existing popular
WebUIs.

Open WebUI Features

- 🤝 **Ollama/OpenAI API Integration:** Integrate OpenAI-compatible APIs & customize API URLs.
- 📄 **Local RAG Integration:** Load and interact with documents using RAG.
- 🔍 **Web Search for RAG:** Perform web searches and inject results into chats.
- 🌐 **Web Browsing:** Integrate web content into chats using URLs.
- 🎨 **Image Generation:** Incorporate image generation using external/local tools.
- ⚙️ **Many Models Conversations:** Engage multiple models simultaneously for diverse responses.
- 🗝️ **RBAC:** Secure access with role-based permissions.
- 🎤 **Voice/Video Call:** Integrated hands-free voice and video communication.
- 🛠️ **Model Builder:** Create and customize Ollama models and chat agents.
- 🐍 **Python Function Calling:** Integrate custom Python functions directly in chat.
- ⚙️ **Pipelines & Plugin Support:** Integrate custom Python logic & plugins for enhanced functionality.
- 🖋️ **Markdown/LaTeX Support:** Full support for rich text and math notation.
- 🌍 **Multilingual Support:** Full internationalization for different languages.

Two Options, One Cloud



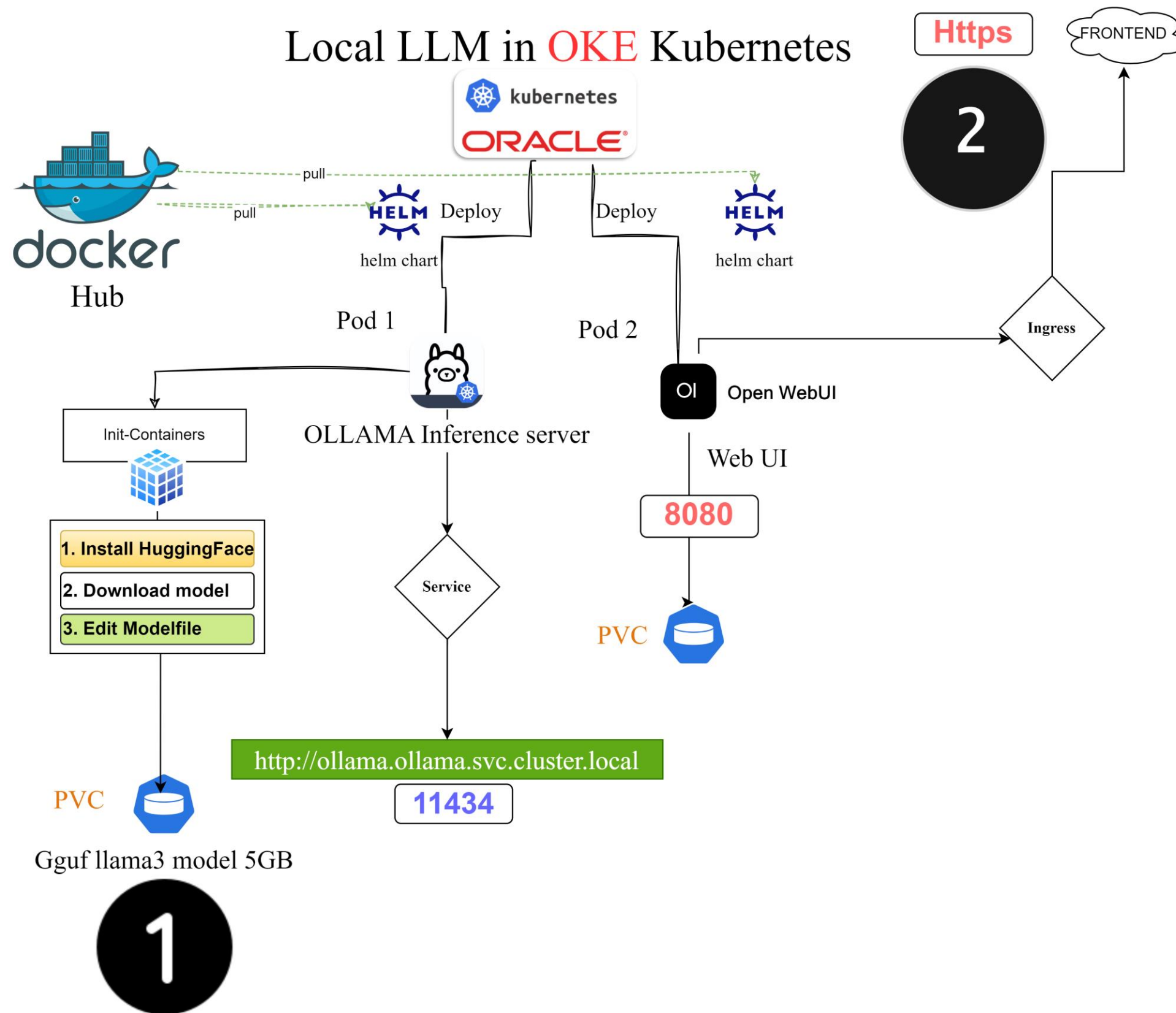
From scratch (Docker to OKE)

- Streamlit for UI
- Python Front end
- Ollama for LLM backend.

Helm to OCI Kubernetes engine

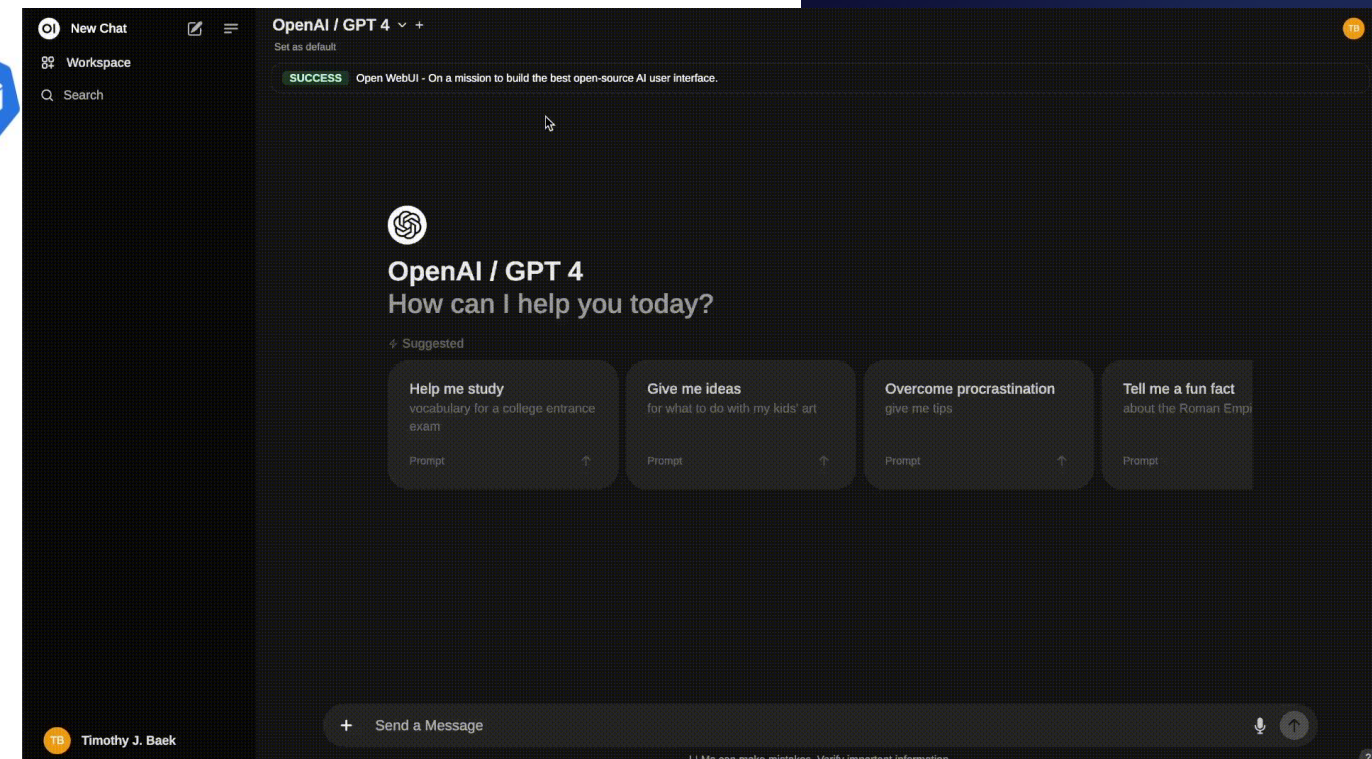
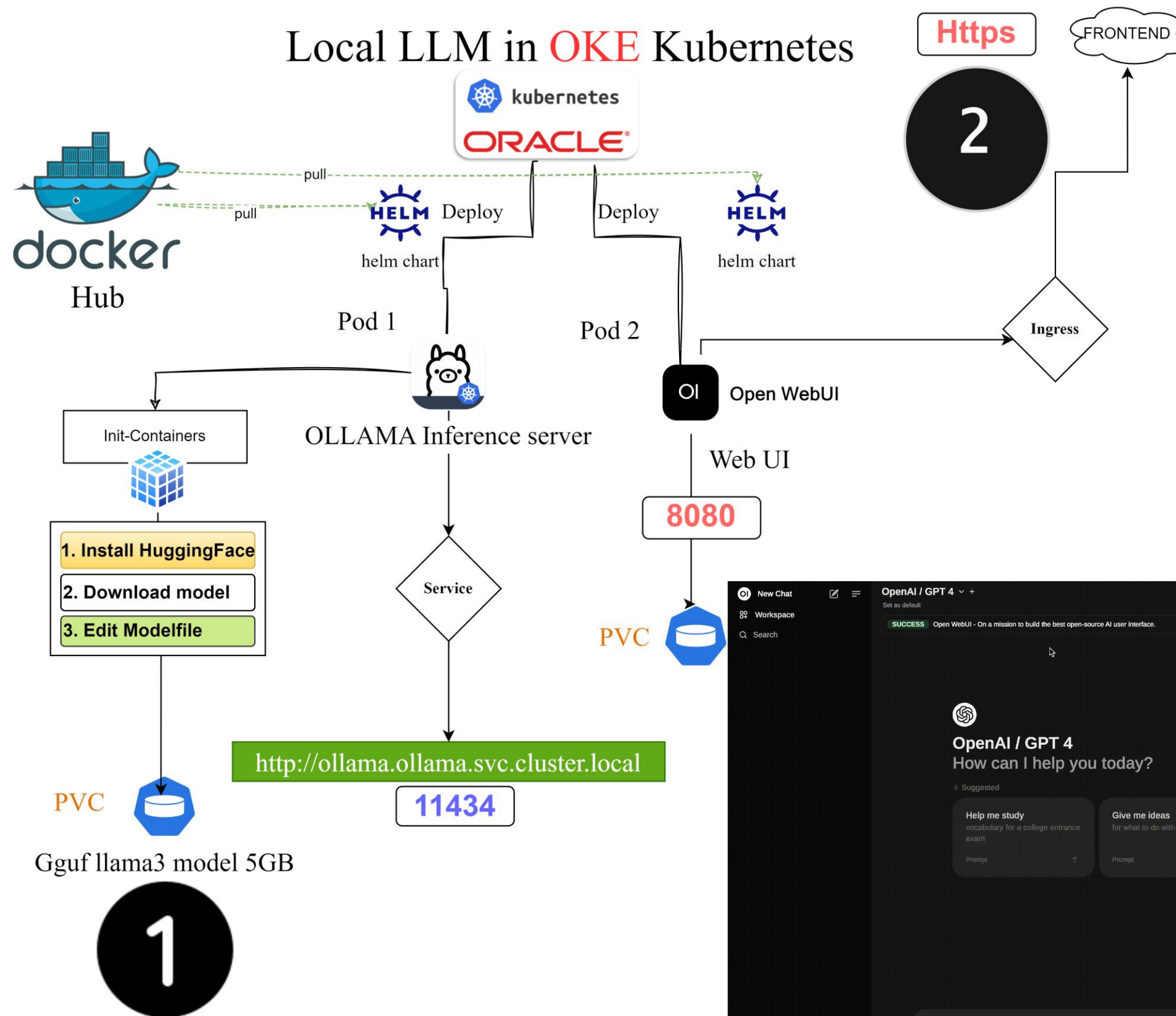
- 2 pods: Helm charts for deployment.
- Ollama backend, initcontainer
- OpenWebUI,

Local LLM in OKE Kubernetes



es engine
ment.

Local LLM in OKE Kubernetes





DEMO

TAKE YOUR

MEMO

ANYWHERE!

OIDC in GitHub actions



THANK YOU



`Tech Beats unplugged`

**Tech Beats
Unplugged**



Cloud Dude
Kosseila

Solutions architect @CloudThrill



Coudthrill.io



Twitter.com/CloudDude_



Linkedin.com/in/kousshd



GitHub.com/brokedba



The Future of Private AI: What's Next?



HELM Leaderboard for Model Safety

The HELM (Holistic Evaluation of Language Models) Leaderboard will play a crucial role in assessing and improving the safety of LLMs. This tool will help organizations track and compare different models based on various safety metrics, ensuring responsible AI development.



Advanced LLM Development Tools

The next wave of AI application building tools will focus on enhancing developer productivity and model customization. Expect to see more sophisticated IDEs, debugging tools, and model fine-tuning platforms tailored for private LLM development.



RAG integration & Function calling

As AI becomes more pervasive, expect increased focus on privacy-preserving AI techniques, federated learning, and robust governance frameworks. These trends will shape the development of more secure and ethically sound AI systems for businesses.

LLM

Demo: From Local to Cloud-Native

Aspect	Local Implementation	OKE Deployment
Tools	Streamlit, Ollama	Helm, Ollama, OpenWebUI, OKE
Objective	Secure, private local chatbot	Scalable, robust cloud deployment
Advantages	Rapid prototyping, full control	High availability, easy scaling
Challenges	Limited scalability, resource constraints	More complex setup, network configuration